

EEUC201 | ANALOG AND DIGITAL ELECTRONICS

Introduction to Digital Electronics and Numbering Systems

Digital Signals • Number Systems • Complements • Binary Arithmetic • Codes

From analog waveforms to binary logic — the foundations of
all digital systems and computer arithmetic

1	0	1	1
0	0	1	1
0	1	0	1
1	0	1	0

What We Will Cover

01

Digital Fundamentals

Digital circuits & signals, the binary number system, signed/unsigned representation

02

Complements & Arithmetic

1's and 2's complement, binary addition/subtraction, multiplication & division

03

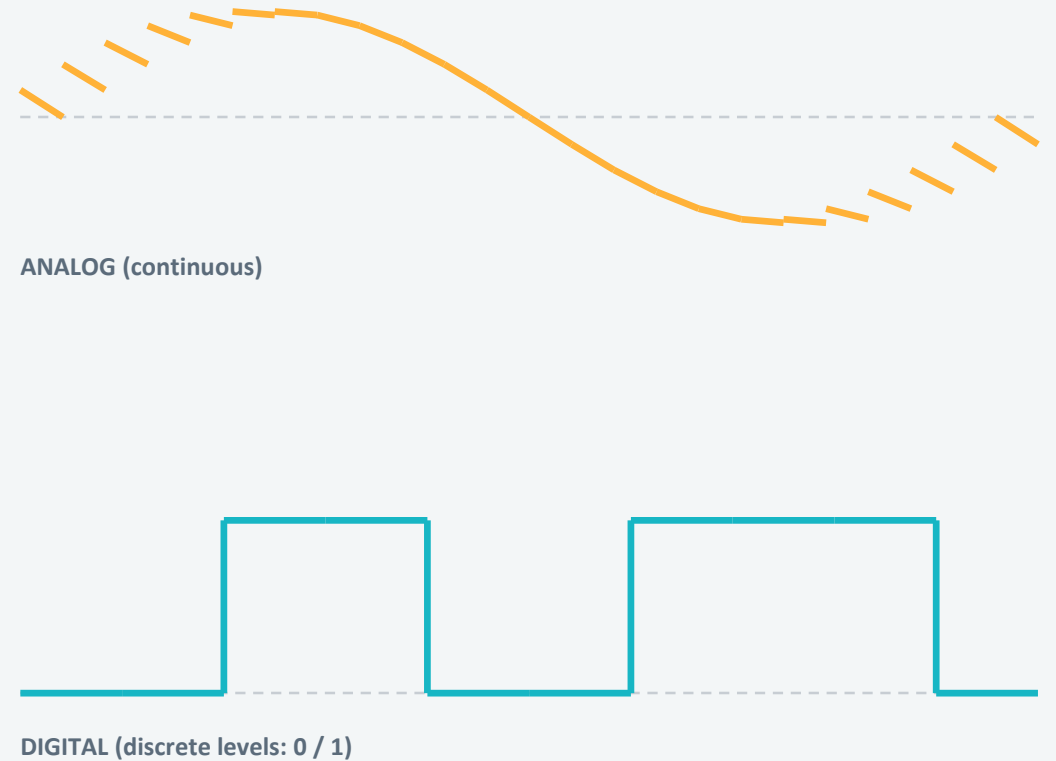
Number Systems & Codes

Octal, hexadecimal, decimal & inter-conversions, BCD numbers, ASCII and EBCDIC codes

Digital Circuits and Digital Signals

- Analog signals vary continuously over time and can take any value within a range
- Digital signals take on only a finite set of discrete levels — most commonly two: LOW (0) and HIGH (1)
- Digital circuits process these discrete levels using switching devices (transistors) operating essentially as ON/OFF switches
- Advantages over analog: better noise immunity, easy storage, accurate reproduction, and compatibility with computer processing
- Every digital system — computers, calculators, mobile phones — is ultimately built from circuits that manipulate 0s and 1s

ANALOG VS DIGITAL WAVEFORM



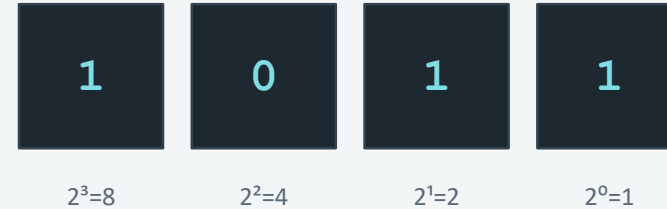
The Binary Number System

- Base (radix) 2 — uses only two digits, 0 and 1, called bits (binary digits)
- Each position represents a power of 2, increasing from right (LSB) to left (MSB)
- Natural fit for digital circuits, since each bit maps directly to an ON/OFF switching state
- A group of 4 bits is a nibble; a group of 8 bits is a byte

DECIMAL VALUE OF A BINARY NUMBER

$$N = \sum (bit_i \times 2^i), i = 0, 1, 2, \dots$$

EXAMPLE: $1011_2 = 11_{10}$



$$\begin{aligned} &1 \times 8 + 0 \times 4 + 1 \times 2 + 1 \times 1 \\ &= 8 + 0 + 2 + 1 \\ &= 11 \text{ (decimal)} \end{aligned}$$

Signed and Unsigned Binary Numbers

- Unsigned representation: all bits represent magnitude only — used when values are always non-negative (e.g. memory addresses, counters)
- An n-bit unsigned number ranges from 0 to $2^n - 1$
- Signed representation: the Most Significant Bit (MSB) is reserved as a sign bit — 0 for positive, 1 for negative
- Common signed formats: Sign-Magnitude, 1's Complement, and 2's Complement (the standard used in virtually all modern digital systems)
- An n-bit 2's complement signed number ranges from -2^{n-1} to $+2^{n-1} - 1$

8-BIT EXAMPLE: SIGN BIT POSITION



RANGE (n = 8 bits)

Unsigned: 0 to 255

Signed (2's complement): -128 to +127

1's Complement

- Obtained by inverting every bit of the binary number — every 0 becomes 1 and every 1 becomes 0
- Used to represent negative numbers: MSB = 1 indicates a negative value
- Drawback: has two representations of zero (+0 = 00000000, -0 = 11111111), which complicates arithmetic
- Subtraction using 1's complement requires an 'end-around carry' correction step

RULE

1's complement of N = invert every bit of N

EXAMPLE: FIND 1'S COMPLEMENT OF 01011010

N

0	1	0	1	1	0	1	0
---	---	---	---	---	---	---	---

invert every bit (0 $\leftrightarrow$ 1)

N'

1	0	1	0	0	1	0	1
---	---	---	---	---	---	---	---

01011010 → 10100101
(each bit flipped)

2's Complement

- Obtained by taking the 1's complement of the number and then adding 1 to the least significant bit
- The standard method for representing negative numbers in virtually all modern processors
- Has a single, unique representation of zero — eliminates the ± 0 ambiguity of 1's complement
- Subtraction becomes simple addition: $A - B$ is computed as $A + (2\text{'s complement of } B)$, discarding any final carry

RULE

$$2\text{'s complement of } N = (1\text{'s complement of } N) + 1$$

EXAMPLE: FIND 2'S COMPLEMENT OF 01011010

N

0	1	0	1	1	0	1	0
---	---	---	---	---	---	---	---

1's comp

1	0	1	0	0	1	0	1
---	---	---	---	---	---	---	---

+ 1

0	0	0	0	0	0	0	1
---	---	---	---	---	---	---	---

2's comp

1	0	1	0	0	1	1	0
---	---	---	---	---	---	---	---

Binary Arithmetic

- Addition rules: $0+0=0$, $0+1=1$, $1+0=1$, $1+1=10$ (write 0, carry 1)
- Subtraction rules: borrow from the next higher bit when subtracting 1 from 0
- Multiplication follows the same shift-and-add process as decimal long multiplication, using AND-like bit products
- Division follows the same repeated subtraction/shift process as decimal long division

ADDITION EXAMPLE

```
  1011   (11)
+ 0110   ( 6)
-----
 10001  (17)
```

SUBTRACTION EXAMPLE (borrow)

```
  1100   (12)
- 0101   ( 5)
-----
  0111   ( 7)
```

MULTIPLICATION EXAMPLE

```
   101   (5)
×  011   (3)
-----
   101
  101
-----
 1111   (15)
```


The Octal Number System

- Base (radix) 8 — uses digits 0 through 7
- Each octal digit corresponds exactly to a group of 3 binary bits, making binary-octal conversion very fast
- Historically used to represent binary data compactly for humans, before hexadecimal became more common
- Position weights are powers of 8: ..., 8^2 , 8^1 , 8^0 (and 8^{-1} , 8^{-2} for fractions)

BINARY ↔ OCTAL

Group binary bits in 3s (from LSB) ↔ 1 octal digit

EXAMPLE: $101\ 110\ 011_2 = 563_8$

1	0	1
---	---	---

= 5

1	1	0
---	---	---

= 6

0	1	1
---	---	---

= 3

$101 = 5$

$110 = 6$

$011 = 3$

$101110011_2 = 563_8$

The Hexadecimal Number System

- Base (radix) 16 — uses digits 0–9 plus letters A–F to represent values 10–15
- Each hex digit corresponds exactly to a group of 4 binary bits (1 nibble)
- Widely used in programming, memory addressing, and color codes because it's compact yet maps cleanly onto binary/bytes
- Position weights are powers of 16: ..., 16^2 , 16^1 , 16^0

A=10 B=11 C=12 D=13 E=14 F=15

EXAMPLE: $1011\ 1100_2 = \text{BC}_{16}$

1	0	1	1
---	---	---	---

= B

1	1	0	0
---	---	---	---

= C

$1011 = \text{B} \ (11)$

$1100 = \text{C} \ (12)$

$10111100_2 = \text{BC}_{16}$

Inter-Conversion of Number Systems

- Decimal $\rightarrow$ Any base: repeated division by the base, collecting remainders (read bottom-to-top)
- Any base $\rightarrow$ Decimal: multiply each digit by its positional weight (power of the base) and sum
- Binary $\leftrightarrow$ Octal / Hex: group bits directly (3 bits $\leftrightarrow$ octal digit, 4 bits $\leftrightarrow$ hex digit) — no arithmetic needed
- Octal $\leftrightarrow$ Hex: convert through binary as an intermediate step

Decimal	Binary	Octal	Hex
25	11001	31	19
100	1100100	144	64
255	11111111	377	FF

Quick-reference conversion table

DECIMAL $\rightarrow$ BINARY: 25 BY REPEATED DIVISION

$$25 \div 2 = 12 \quad \text{rem } 1$$

$$12 \div 2 = 6 \quad \text{rem } 0$$

$$6 \div 2 = 3 \quad \text{rem } 0$$

$$3 \div 2 = 1 \quad \text{rem } 1$$

$$1 \div 2 = 0 \quad \text{rem } 1$$

Read remainders bottom $\rightarrow$ top:

$$25_{10} = 11001_2$$

BCD Numbers (Binary Coded Decimal)

- Each decimal digit (0–9) is represented independently by its own 4-bit binary code
- Unlike pure binary, BCD does not convert the whole number — every digit is encoded separately
- Simplifies interfacing with decimal displays (7-segment displays, digital calculators) since each digit converts directly
- Less efficient than pure binary for arithmetic — invalid 4-bit codes 1010–1111 (10–15) are never used in BCD

VALID BCD RANGE PER DIGIT

0000 (0) to 1001 (9) only

EXAMPLE: DECIMAL PRESENT AS BCD

digit 5

0	1	0	1
---	---	---	---

digit 2

0	0	1	0
---	---	---	---

digit 9

1	0	0	1
---	---	---	---

Decimal:	5	2	9
BCD:	0101	0010	1001

529 (decimal) → 0101 0010 1001 (BCD)
(Note: pure binary of 529 is completely different:
1000010001)

ASCII Code

- American Standard Code for Information Interchange — the most widely used character-encoding scheme
- Standard ASCII uses 7 bits, giving 128 possible codes (0–127) for letters, digits, punctuation, and control characters
- Extended ASCII uses 8 bits (256 codes), adding extra symbols and accented characters
- Uppercase and lowercase letters, digits 0–9, and common symbols each have a fixed, standardized code
- Universally used for text representation in computers, communication protocols, and data storage

SAMPLE ASCII CODES

Character	Decimal	Binary (7-bit)
'A'	65	1000001
'a'	97	1100001
'0'	48	0110000
Space	32	0100000
Enter (CR)	13	0001101

Note: 'a' = 'A' + 32 — lowercase and uppercase letters differ by a fixed offset in ASCII

EBCDIC Code

- Extended Binary Coded Decimal Interchange Code — an 8-bit character-encoding scheme developed by IBM
- Used primarily on IBM mainframe and midrange computer systems, unlike ASCII which dominates on PCs and the internet
- Provides 256 possible codes, but the assignment of codes to characters is different from ASCII — the two are not directly compatible
- Letters are not encoded as one continuous block — there are gaps in the code sequence between certain letter ranges, a historical quirk of its design
- Data exchanged between EBCDIC-based mainframes and ASCII-based systems must be explicitly translated between the two codes

ASCII VS EBCDIC — KEY DIFFERENCES

Feature	ASCII	EBCDIC
Bit width	7 (std.) / 8	8
Max codes	128 / 256	256
Common use	PCs, Internet	IBM mainframes
Letter sequence	Continuous	Has gaps

SUMMARY

From Signals to Symbols: The Language of Digital Systems

Digital Fundamentals

- Digital circuits & signals
- Binary number system
- Signed vs unsigned numbers

Complements & Arithmetic

- 1's complement
- 2's complement
- Addition, subtraction, mult., div.

Systems & Codes

- Octal & hexadecimal
- Inter-conversions
- BCD, ASCII, EBCDIC